

NiceGUI ui.timer 全面详解

NiceGUI 的 `ui.timer` 是用于定期执行回调函数的核心组件，设计初衷是为了简化界面的定时更新场景（如实时数据图表、倒计时、动态状态刷新等）。它支持灵活的激活 / 停用、单次执行、延迟启动等功能，且可在运行时动态调整关键参数，适用于各类需要定时触发逻辑的 UI 开发场景。

一、核心概念与初始化参数

`ui.timer` 通过指定时间间隔和回调函数创建定时任务，初始化时支持 5 个核心参数，部分参数可在运行时动态修改：

参数名	类型	说明
<code>interval</code>	数值（秒）	回调函数的执行间隔（支持运行时修改，如 <code>timer.interval = 2.0</code> ）
<code>callback</code>	函数或协程（coroutine）	时间间隔结束时执行的逻辑（支持同步函数和异步协程，如 <code>async def</code> 定义的函数）
<code>active</code>	布尔值（默认：True）	定时器是否激活（支持运行时修改，如 <code>timer.active = False</code> 暂停）
<code>once</code>	布尔值（默认：False）	是否仅执行一次（True 时为“延迟执行”，执行后自动终止）
<code>immediate</code>	布尔值（默认：True）	是否立即执行第一次回调（v2.9.0 新增， <code>once=True</code> 时该参数失效）

关键点：

- 若 `immediate=True`（默认）：定时器创建后立即执行一次回调，之后按 `interval` 循环；
- 若 `immediate=False`：首次回调会延迟 `interval` 秒后执行，后续按间隔循环；
- `callback` 支持异步协程（需用 `async def` 定义），可用于处理网络请求、耗时计算等场景（需配合 `asyncio.sleep()` 而非 `time.sleep()`）。

二、基础使用场景与示例

1. 实时更新 UI（如显示当前时间）

最常见场景：每秒刷新标签显示当前时间，默认立即执行。

```
from datetime import datetime
from nicegui import ui

label = ui.label() # 创建空标签
# 1秒间隔，每次执行更新标签文本为当前时间（%X 表示时分秒）
ui.timer(1.0, lambda: label.set_text(f'{datetime.now():%X}'))

ui.run()
```

2. 延迟执行单次任务（once 参数）

通过 `once=True` 实现“延迟 N 秒后执行一次”，适用于提示、超时触发等场景：

```
from nicegui import ui

def handle_click():
    # 延迟1秒后弹出通知，仅执行一次
    ui.timer(1.0, lambda: ui.notify('延迟1秒的通知! '), once=True)

# 点击按钮触发延迟任务
ui.button('点击后1秒提示', on_click=handle_click)

ui.run()
```

3. 延迟启动定时器（immediate 参数）

默认 `immediate=True` 会立即执行第一次回调，设置为 `False` 可让首次执行延迟 `interval` 秒：

```
from datetime import datetime
from nicegui import ui

label = ui.label()
# 首次执行延迟1秒，之后每秒更新一次时间
ui.timer(1.0, lambda: label.set_text(f'{datetime.now():%X}'), immediate=False)

ui.run()
```

三、高级操作：激活、停用与取消

`ui.timer` 支持动态控制生命周期，包括激活 / 停用、永久取消，以及取消当前正在执行的回调。

1. 激活与停用（active 属性）

通过修改 `active` 属性可暂停或恢复定时器，适用于需要临时启停的场景（如开关控制）：

```

from nicegui import ui

# 创建滑块，初始值0.5
slider = ui.slider(min=0, max=1, value=0.5)
# 0.1秒间隔，滑块值每次增加0.01（循环0-1）
timer = ui.timer(0.1, lambda: slider.set_value((slider.value + 0.01) % 1.0))

# 开关绑定定时器的 active 属性，控制启停
ui.switch('启用定时器').bind_value_to(timer, 'active')
# 按钮永久取消定时器（取消后无法再激活）
ui.button('永久取消', on_click=timer.cancel)

ui.run()

```

2. 永久取消定时器（cancel 方法）

调用 `timer.cancel()` 会彻底终止定时器，之后无法通过 `active` 属性重新激活。

3. 取消当前执行的回调（with_current_invocation 参数）

v2.23.0 新增 `with_current_invocation` 参数，支持取消“正在执行中的回调任务”（如异步协程、循环任务）：

```

import asyncio
from nicegui import ui

# 创建进度条
progress = ui.linear_progress().props('instant-feedback')

# 异步协程：2秒内逐步填充进度条
async def cycle_once():
    for i in range(10):
        progress.value = (i + 1) / 10 # 每次增加10%
        await asyncio.sleep(0.2) # 异步等待0.2秒

def start_progress():
    # 2.5秒间隔重复执行进度条逻辑，立即启动
    timer = ui.timer(2.5, cycle_once, immediate=True)
    # 控制按钮组
    with ui.column() as controls:
        # 仅取消定时器，当前进度条循环继续执行完
        ui.button('取消定时器（当前任务完成）') \
            .on('click', lambda: timer.cancel(with_current_invocation=False)) \
            .on('click', controls.delete)
        # 取消定时器并终止当前进度条循环
        ui.button('取消定时器（终止当前任务）') \
            .on('click', lambda: timer.cancel(with_current_invocation=True)) \
            .on('click', controls.delete)

# 启动进度条任务
ui.button('开始进度条', on_click=start_progress).props('flat')
ui.run()

```

四、全局定时器 (app.timer)

`ui.timer` 是**页面级定时器**（仅在当前页面上下文生效，页面刷新后重建），而 `app.timer` 是**全局定时器**（独立于 UI 页面，整个应用生命周期内生效），适用于跨页面共享的定时逻辑（如全局计数器、后台数据同步）。

示例：全局计数器（跨页面共享）

```
from nicegui import app, ui

# 全局变量（跨页面共享）
counter = {'value': 0}

# 全局定时器：每秒更新计数器（独立于页面）
app.timer(1.0, lambda: counter.update(value=counter['value'] + 1))

# 页面1：显示计数器值
@ui.page('/')
def page1():
    ui.label('全局计数器: ').bind_text_from(counter, 'value', lambda v: f'当前值: {v}')

# 页面2：同样显示全局计数器（共享同一值）
@ui.page('/page2')
def page2():
    ui.label('页面2的计数器: ').bind_text_from(counter, 'value', lambda v: f'当前值: {v}')

ui.run()
```

关键区别：

特性	ui.timer (页面级)	app.timer (全局级)
生效范围	仅当前页面	整个应用（跨页面共享）
生命周期	页面刷新 / 关闭后销毁	应用启动后持续运行，直到应用停止
依赖 UI	依赖页面上下文（需在页面内创建）	不依赖 UI（可在页面外创建）

五、核心属性与方法

1. 可绑定属性（支持动态修改）

属性名	类型	说明
<code>active</code>	布尔值	控制定时器激活 / 停用（可通过 <code>bind_value_to</code> 绑定到开关等组件）
<code>interval</code>	数值	动态修改执行间隔（如 <code>timer.interval = 3.0</code> 改为 3 秒）
<code>visible</code>	布尔值	控制定时器关联的 UI 元素可见性（继承自 Element）

2. 常用方法

方法名	参数	说明
<code>activate()</code>	-	激活定时器（等价于 <code>timer.active = True</code> ）
<code>deactivate()</code>	-	停用定时器（等价于 <code>timer.active = False</code> ）
<code>cancel(with_current_invocation=False)</code>	<code>with_current_invocation</code> ：是否取消当前执行的回调	永久取消定时器（取消后不可恢复）
<code>bind_visibility_from(target, name)</code>	<code>target</code> ：目标对象， <code>name</code> ：属性名	绑定可见性到目标对象的属性（如开关控制显示）
<code>update()</code>	-	强制更新定时器关联的 UI 元素
<code>delete()</code>	-	删除定时器及关联的 UI 元素

六、注意事项与最佳实践

- 异步回调处理**：若回调函数是异步协程（`async def`），需使用 `asyncio.sleep()` 而非 `time.sleep()`，避免阻塞事件循环。
- 资源释放**：页面关闭时，`ui.timer` 会自动销毁，无需手动取消；但 `app.timer` 需在应用停止时手动取消（若需提前终止），避免内存泄漏。
- 高频定时器**：若 `interval` 过短（如 <0.1 秒），需注意回调函数的执行效率，避免 UI 卡顿（建议耗时操作放在异步回调中）。
- 版本兼容性**：`immediate` 参数和 `app.timer` 新增于 v2.9.0，`with_current_invocation` 新增于 v2.23.0，使用时需注意 NiceGUI 版本。
- 绑定数据**：通过 `bind_text_from`、`bind_value_to` 等方法，可将定时器的属性与 UI 组件关联（如滑块控制 `interval`，开关控制 `active`）。

七、总结

`ui.timer` 是 NiceGUI 中处理定时任务的核心组件，具备以下优势：

- 简单易用：一行代码创建定时任务，支持同步 / 异步回调；
- 灵活控制：支持激活 / 停用、动态修改间隔、取消当前任务等；
- 场景覆盖：从页面级 UI 实时更新到全局后台任务，均可满足；
- 无缝集成：与 NiceGUI 的数据绑定、组件系统深度兼容，开发效率高。

适用于实时数据展示、倒计时、轮播图、后台同步、定时通知等各类场景，配合 `app.timer` 可实现跨页面的全局定时逻辑，是 NiceGUI 开发中不可或缺的工具。